

Security Authority Management System (React + Express)

A modern, full-stack web application with secure authentication, built using React (Frontend), Express.js (Backend), and MySQL (Database). This project demonstrates a clean architecture with secure JSON Web Token (JWT) sessions and hashed passwords.

How to Run This Project (If downloaded as a ZIP)

If you received this project as a ZIP file, follow these exact steps to get it running on your local machine:

1. Prerequisites

Before you start, make sure you have the following installed:

- [Node.js](#) (Version 16+ recommended)
- [MySQL Server](#) running locally on your computer.

2. Database Setup

1. Open your MySQL client (e.g., MySQL Workbench or the command line).
2. Open and run the `database.sql` file provided in the root folder.
 - This will create a database named `Ankit`.
 - It will create a `users` table with `id`, `name`, `email`, and `password` columns.
3. Open `server.js` and verify that the database credentials match your local MySQL setup (around line 33):
 - **host:** `localhost`

- **user:** root
- **password:** root
- **database:** Ankit

3. Install Dependencies & Start the Backend

1. Extract the ZIP file and open the project folder in your terminal.
2. Install the backend dependencies by running:

```
npm install
```

3. Start the Express backend server (it will run on `http://localhost:3000`):

```
npm run dev
```

(Keep this terminal window open!)

4. Install Dependencies & Start the Frontend

1. Open a **second, new terminal window**.
2. Navigate into the `client` folder:

```
cd client
```

3. Install the frontend dependencies by running:

```
npm install
```

4. Start the React frontend server:

```
npm run dev
```

5. Open your browser and go to the link shown in the terminal (usually `http://localhost:5173`). You can now register a user and log in!
-

Project Structure Explained

Here is a breakdown of what every file and folder in this project does:

Root Directory (Backend API)

- `server.js` : The heart of the backend. It sets up the Express server, connects to MySQL, handles user registration/login (with password hashing and JWT creation), and provides the secure API routes for the React app to consume.
- `database.sql` : Contains the raw SQL queries needed to set up the database and the table schema required for the application to function.
- `package.json` : Keeps track of the backend project details and all the NPM dependencies installed (like `express` , `mysql2` , `bcrypt` , `jsonwebtoken` , and `cors`).
- `package-lock.json` : Automatically generated file that locks the exact versions of the dependencies installed to ensure the project runs the exact same way on anyone else's computer.
- `README.md` : The instruction manual you are currently reading!

`client/` Directory (React Frontend)

- `client/package.json` : Keeps track of the React frontend dependencies (like `react` , `react-router-dom` , and `axios`).
- `client/index.html` : The single HTML file that serves the entire React application.
- `client/src/main.jsx` : The main entry point of the React app that connects React to the `index.html` file.
- `client/src/index.css` : Contains all the global styling, colors, and layout rules for the website.
- `client/src/App.jsx` : The master component that controls the Routing/Navigation between different pages (Login, Register, and Dashboard).
- `client/src/components/Login.jsx` : The login page component. It sends user credentials to the backend and saves the JWT token upon success.

- `client/src/components/Register.jsx` : The registration page component. It allows new users to sign up and have their passwords securely hashed.
- `client/src/components/Dashboard.jsx` : The main protected application page. It securely fetches the list of authorities using the saved JWT token, and provides forms to Add, Edit, or Delete records.